

See discussions, stats, and author profiles for this publication at: <https://www.researchgate.net/publication/236952142>

High Performance GPU Accelerated Local Optimization in TSP

Conference Paper · May 2013

DOI: 10.1109/IPDPSW.2013.227

CITATIONS

16

READS

786

2 authors:



Kamil Rocki

The University of Tokyo

12 PUBLICATIONS 141 CITATIONS

[SEE PROFILE](#)



Reiji Suda

The University of Tokyo

96 PUBLICATIONS 837 CITATIONS

[SEE PROFILE](#)

High Performance GPU Accelerated Local Optimization in TSP

Kamil Rocki, Reiji Suda

The University of Tokyo

Graduate School of Information Science and Technology

Department of Computer Science

7-3-1, Hongo, Bunkyo-ku, Tokyo, Japan

Email: {kamil.rocki, reiji}@is.s.u-tokyo.ac.jp

Abstract—This paper presents a high performance GPU accelerated implementation of 2-opt local search algorithm for the Traveling Salesman Problem (TSP). GPU usage significantly decreases the execution time needed for tour optimization, however it also requires a complicated and well tuned implementation. With the problem size growing, the time spent on local optimization comparing the graph edges grows significantly. According to our results based on the instances from the TSPLIB library, the time needed to perform a simple local search operation can be decreased approximately 5 to 45 times compared to a corresponding parallel CPU code implementation using 6 cores. The code has been implemented in OpenCL and as well as in CUDA and tested on AMD and NVIDIA devices. The experimental studies show that the optimization algorithm using the GPU local search converges from up to 300 times faster compared to the sequential CPU version on average, depending on the problem size. The main contributions of this paper are the problem division scheme exploiting data locality which allows to solve arbitrarily big problem instances using GPU and the parallel implementation of the algorithm itself.

Keywords—Parallel Architectures, Optimization, Optimal Scheduling, GPU Computing.

I. INTRODUCTION

A. Traveling salesman problem

The traveling salesman problem (TSP)[1][2][3] is one of the most widely studied combinatorial optimization problems. It has become a benchmark for new algorithms since it provides a set of standard problems making it possible to compare the results among the published works. This problem is classified as NP-hard[4], with a pure brute force algorithm requiring factorial execution time. In the symmetric TSP, the distance between two cities is the same in each opposite direction, forming an undirected graph, therefore the number of possible solutions is halved. The most direct solution for a TSP problem is to permute all the possible tours through n cities. Given a starting city, there can be $n-1$ choices for the second city, and $n-2$ choices for the third city accordingly. In the asymmetric TSP, paths may not exist in both directions or the distances might be different, forming a directed graph. A number of heuristics have been developed which return reasonably good approximations to the optimal tour in polynomial time.

One of the most well known methods of approaching the problem is repeating a series of steps called 2-opt exchanges [5]. According to our results, at least 90% of the execution time during the Iterated Local Search (ILS)[11] (which we are using and this algorithm is a part of - Algorithm 1) is spent on the 2-opt checks/exchanges and that number strongly increases with the problem size growing. This is easy to explain as the complexity of the 2-opt search is $O(n^2)$ and any simple perturbation method would be of order $O(n)$, where n is the number of the cities. In order to solve the whole problem quickly using the iterative approach, this basic step has to be optimized. Recently, the use of graphics processing units (GPUs) as general-purpose computing devices has risen substantially, accelerating many non-graphics programs that exhibit a lot of parallelism with low synchronization requirements. In addition to that, the current trend in modern supercomputer architectures is to use GPUs as low-power consumption devices. Therefore, it is important to analyze possible application of CUDA on OpenCL as GPU programming languages to speedup optimization and provide a basis for future similar applications using massively parallel systems. The scope of this paper is to present the parallel 2-opt algorithm and the optimizations required to achieve high performance on GPU. An important thing that has to be mentioned is that this paper focuses only on the 2-opt local search, not the ILS algorithm, therefore the reported results are refer to a single 2-opt run.

Algorithm 1 Generic Iterated Local Search, Highlighted Parts are CUDA/OpenCL accelerated

```

1: procedure ITERATED LOCAL SEARCH
2:    $s_0 \leftarrow \text{GenerateInitialSolution}()$ 
3:    $s^* \leftarrow \text{2optLocalSearch}(s_0)$ 
4:   while termination condition not met do
5:      $s' \leftarrow \text{Perturbation}(s^*)$ 
6:      $s^{*'} \leftarrow \text{2optLocalSearch}(s')$ 
7:      $s^* \leftarrow \text{AcceptanceCriterion}(s^*, s^{*'})$ 
8:   end while
9: end procedure

```

B. 2-opt

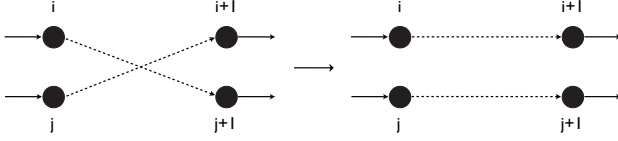


Figure 1. A 2-opt move

The 2-opt algorithm is one of the simplest TSP optimization methods. It removes two edges from the tour, and reconnects the two paths created, this is often referred to as a 2-opt move. There is only one possible way to reconnect the two paths so that the tour remains valid (Fig. 2). This step is applied only if the new tour would be shorter. Repeating removing and reconnecting the tour until no 2-opt improvements can be found leads to an optimal route (local minimum). It improves tour by reconnecting and reversing order of sub-tour. For example, every pair of edges:

$$[i, j + 1] \text{ and } [j, i + 1]$$

is checked if an improvement is possible $[d(x,y)$ is distance between x and y]:

$$d(i, i + 1) + d(j, j + 1) < d(i, j + 1) + d(j, i + 1)$$

$$\text{distance}(\mathbf{B}, \mathbf{F}) + \text{distance}(\mathbf{G}, \mathbf{D}) > \text{distance}(\mathbf{B}, \mathbf{D}) + \text{distance}(\mathbf{G}, \mathbf{F})$$

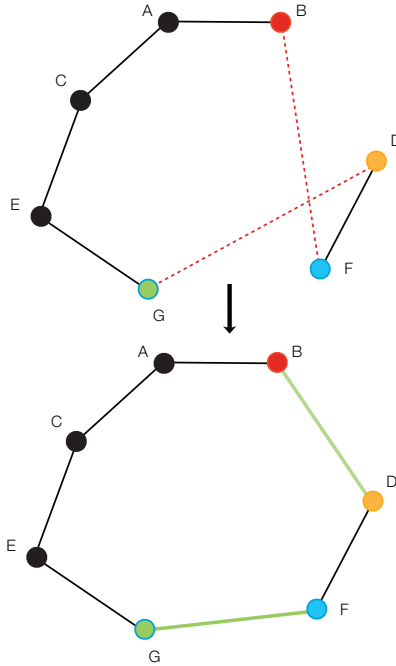


Figure 2. An example of a 2-opt step

The procedure is repeated until no further improvement can be done. It is good for finding a local optimum (a solution that cannot be improved by considering a neighboring configuration) but it is not guaranteed to find the best possible solution (the global optimum) out of all possible solutions [Rego and Glover 2002]. In order to find the global solution the algorithm needs a modification. A method allowing *escaping* from local optima has to be provided, which usually means that a local solution needs to be worsened in some way, keeping it within a certain search-space.

II. PROBLEM STATEMENT

A. Parallelization

The main problem of the GPU implementation is the work division scheme. With thousands of threads running simultaneously, the algorithm has to be simple, yet efficient enough to maintain scalability. GPU threads are very lightweight and best suited for short and simple task such as pixel processing in an image.

B. Memory limitation

There are two basic methods of obtaining the distance between 2 points. The first one, used very often to avoid unnecessary computation by data reuse utilizes a Look Up Table (LUT). It means that the distances are pre-computed and later read from memory. The main disadvantage of this approach is that it requires $O(n^2)$ space (Table I). That is especially problematic when GPUs are considered. The other way is to read coordinates of the points only and calculate the distance each time it is needed. The space required for this task is only $O(n)$ which exploits data locality and bigger portions of a problem can fit into cache. Due to high memory limitations and abundant compute power, GPUs are more suited to do the latter. Currently, a typical GPU is equipped with approximately 1-3GB of relatively slow (latency-wise) global memory, approximately 32kB to 64kB of much faster on-chip local memory or cache and peak computational power of over 1 TFLOP/s.

Table I
2-OPT SINGLE RUN - MEMORY NEEDED

Problem (TSPLIB)	Number of cities (points)	Memory needed for LUT (MB)	Memory needed for coords (kB)
kroE100	100	0.038	0.78
ch130	130	0.065	1.02
ch150	150	0.086	1.17
kroA200	200	0.15	1.56
ts225	225	0.19	1.75
pr299	299	0.34	2.34
pr439	439	0.74	3.43
rat783	783	2.34	6.12
vm1084	1084	4.48	8.47
pr2392	2392	21.8	18.69
pcb3038	3038	35.21	23.73
fnl4461	4461	75.9	34.85

III. RELATED WORKS

The factorial algorithm's complexity motivated the research on two types of algorithms, exact algorithms and heuristics algorithms. The exact algorithms search for an optimal solution through the use of branch-and-bound, linear programming or branch-and-bound plus cut based on linear programming[12] techniques. These methods are usually hard to parallelize. Heuristics solutions are approximation algorithms that reach an approximate solution (close to the optimal) in a time fraction of the exact algorithm. TSP heuristics algorithms might be based on genetic and evolutionary algorithms[13], simulated annealing [14], Tabu search, neural networks, ant systems, among others. Constructive multi-start search algorithms, such as IHC - iterative hill climbing, are often applied to combinatorial optimization problems like TSP.

These algorithms generate an initial solution and then attempt to improve it using heuristic techniques until a locally optimal solution, for example, one that cannot be further improved, is reached. O'Neil et al.[6] describe and evaluate a parallel implementation of iterative hill climbing with random restart for determining high-quality solutions to the traveling salesman problem. In our opinion and based on our results, an algorithm performing iterative refinement such as ours or GA-based, is a much better solution. This is the reason why we decided to keep the original ILS algorithm and parallelize the 2-opt search itself which is responsible for most of the running time, hopefully leading to strong-scaling. Most of the other works related to parallel TSP solvers involves evolutionary and genetic programming, such as Ant Colony Optimization (ACO)[7] or Genetic Algorithms (GA)[8]. The last work presents a very fast GA algorithm implemented on GPU which clearly outperforms the CPU counterpart. Although it is very fast, the main drawback is that there seems to exist a problem size limitation caused by the memory restrictions. In our opinion, our work is complementary ours, as we do not parallelize the algorithm itself, but the local optimization that can be used by other algorithms.

Another approach to parallel GPU optimization is presented by Luong et al.[20] using local search metaheuristics. However it is hard to compare the results as that paper reports only speedups versus sequential CPU execution. In addition to that, the GPUs which are used are significantly different. In general, there are many ongoing attempts of approaching the problem of GPU TSP optimization, with the best results utilizing ACO and GA. Although this paper presents an algorithm which might not compete with the best state-of-the-art TSP solvers such as Concorde[19], to our best knowledge it is the fastest GPU accelerated local optimization algorithm at the moment.

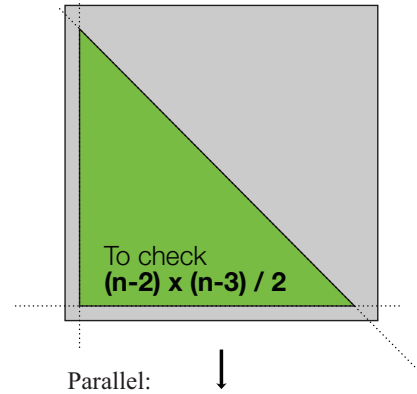
IV. PROPOSED SOLUTION

Assuming that there are N cities in a route, the number of distinct pairs of edges can be approximated by:

$$(N - 3) * (N - 2) / 2,$$

Sequential:

```
for (int i = 1 ; i < n - 2; i++)
  for (int j = i + 1 ; j < n - 1 ; j++)
```



Parallel:

Matrix of city pairs to be checked for a possible swap, each pair corresponds to one GPU job

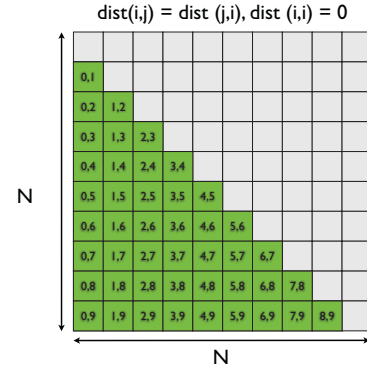


Figure 3. Parallelization scheme

Algorithm 2 An overview of the basic proposed algorithm

- 1: Copy the tour and the coordinates to the GPU global memory (CPU)
 - 2: Execute the kernel (GPU program)
 - 3: Copy the tour and coordinates to the shared memory (fast local memory)
 - 4: Calculate swap effect of one pair (per thread)
 - 5: Find the best value and store it in the global memory
 - 6: Read the result (CPU)
-

```

1 int distance(int i, int j, float2 *coords) {
2     // coordinates stored in fast shared memory in case of GPU execution
3     float dx, dy;
4     dx = coords[i].x - coords[j].x;
5     dy = coords[i].y - coords[j].y;
6     return (int)(sqrtf(dx * dx + dy * dy) + 0.5f); // return Euclidean distance
7 }

```

Listing 1. A simple function calculating Euclidean distance for GPU or CPU between points i and j

which means that i.e. in case of kroE100 100-city problem from TSPLIB, there are 4851 swaps to be checked, 95703 in case of pr439 439-city or 2857245 in case of pr2392 2392-city problem. Thousands of threads can run on a GPU simultaneously and best results are achieved, when GPU is fully loaded to hide the memory latency. Therefore we decided first to dedicate one thread to one 2-opt swap calculation. The procedure is relatively simple. The GPU kernel receives the coordinates of the points/cities and calculates the change in distance by swapping 2 assigned edges using 2D euclidean metric. Using atomic operations the best candidates for swapping are stored in the global memory and are later accessible by CPU.

Each thread has to check the following condition and update the best edges found so far (Fig. 1 and Fig. 2):

$$d(i, i+1) + d(j, j+1) < d(i, j+1) + d(j, i+1)$$

A. Solving small instances

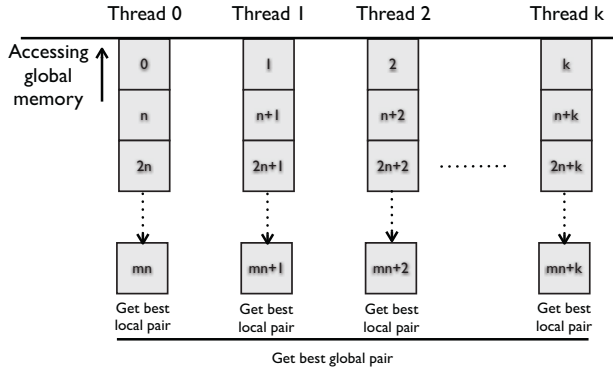


Figure 4. GPU parallel approach, n - total number of threads, $k < n$, $mn + k \geq$ total number of checked pairs $\geq mn$, $\{1, 2, \dots, k\}$ - thread id

1) *Optimization 1: Access pattern - Reusing shared memory:* The easiest approach would be to dedicate one thread to one 2-opt swap calculation. Since the large off-chip GPU memory has high latency and the fast on-chip memory is very limited, accessing pre-calculated data is not a good idea. Additionally, GPU has very high peak computational power compared to CPUs. Therefore the algorithm stores only the coordinates of the points in the fast on-chip 48kB of shared memory (however that limits us to 6144 cities¹)

¹6144 cities: 49152 B (6144 x sizeof(float) x 2) for the coordinates

and calculates the distance each time when it is needed. As in the CPU approach, it calculates first the number of checks performed by each thread: $iter = \frac{n*(n-1)}{2*blocks*threads}$. Then each thread checks assigned cell number and then jumps $blocks*threads$ distance $iter$ times. This allows avoiding excessive global memory usage as it is accessed only one time at the beginning of the execution (Fig. 4). I.e. For a 28 x 1024 configuration (CUDA blocks x threads) and pr2392 problem, $ceil(2857245/(28 * 1024)) = 100$ iterations will be necessary for each thread. That means that each thread will reuse previously stored data in the shared memory 99 times without having to access the slow global memory.

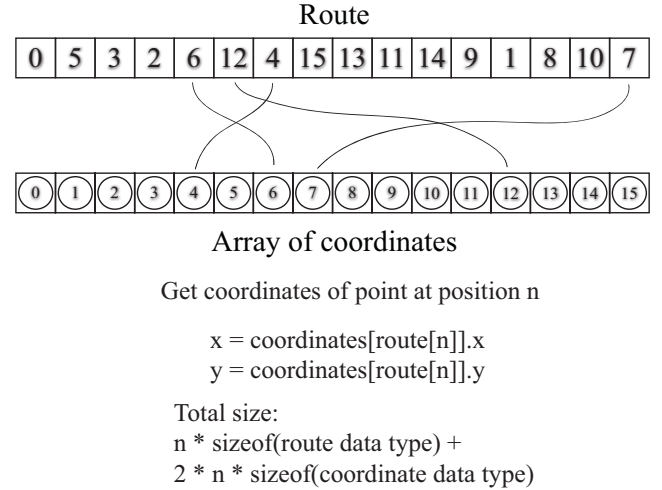


Figure 5. A simple way to obtain point information; pre-loaded coordinates are accessed via the route array

2) *Optimization 2: Access pattern - ordering the coordinates:* Preordering the coordinates (Fig. 5 and Fig. 6) brings 4 benefits:

- 1) No extra address offset calculation - faster
- 2) No route data required - less memory
- 3) Data is ordered when read by GPU - faster, can be read sequentially, no bank conflicts
- 4) Data is ordered - Can be split into subproblems, a basis for the general algorithm

This optimization works by reading the coordinates in the route's order on host before copying the data to the GPU. Then, they are stored in an intermediate array in that

order. This array is then copied to the GPU. It brings some performance degradation caused by the additional time spent on host ($O(n)$), however saves much more time by avoiding scattered read on GPU.

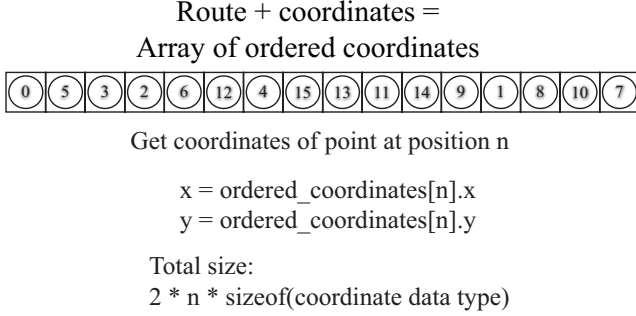


Figure 6. An array containing coordinates in the route's order

B. Solving any instance

Exploiting the fact that the coordinates are ordered, it was possible to overcome the problem size limitation without sacrificing the performance by the following division scheme. Assuming that the GPU's local memory is limited and can store up to m coordinates, we propose the following approach. As presented in Fig. 7, a kernel reads coordinates of the cities from tour ranges $m/2+1$ to m and $N-m/2+1$ to N at one time. Therefore 2 coordinates' ranges are needed, which implies that the maximum subproblem size cannot be larger than $3072 \left(\frac{48kB}{2 \times 2 \times \text{sizeof(float)}} \right)$. Since the problem is divided into several kernel launches, they can be executed independently in a parallel manner.

V. RESULTS AND ANALYSIS

The algorithm has been tested on a GeForce GTX 680 GPU, Radeon HD 7970, Intel Core i7-3960X CPU (with PCIe 3.0) using CUDA 5, AMD OpenCL v. 1.2 and Intel OpenCL v. 1.2 platforms. The table (Table II) presents a comparison of time needed to perform full 2-opt search using different TSPLIB instances. The GPU/CPU comparison is shown in Fig. 10. We used auto-parallelization provided by OpenCL and auto-vectorization features. CUDA and OpenCL implementations are much much faster even after including the time needed for data transfer, whose proportion to the calculation part decreases with the problem size growing. This can be explained by the very fast on-chip shared memory which can be treated as an explicitly managed cache and much higher peak memory throughput compared to the CPU. We believe that memory bandwidth is the limit in case of the parallel CPU implementation. Additionally, due to large data arrays being accessed randomly, cache efficiency is decreased drastically. We recorded the peak GPU performance of 680 GFLOP/s (GeForce using CUDA) and 830 GFLOP/s (Radeon in OpenCL) during the 2-opt optimization (Fig. 9). The last 3 columns show the

time needed from an initial solution based on the Multiple Fragment(Greedy) heuristic[18] to the local minimum found by the algorithm. Up to pla7397, the algorithm reaches very good solutions in less than a second. The main disadvantage of the presented algorithm is that it still requires a significant amount of time to reach the first local minimum in case of larger instances. The solutions to this problem are more sophisticated algorithms such as 3-opt, k-opt or LK. Possibly limiting the neighborhood would also bring an improvement in efficiency.

We have also implemented the Iterated Local Search algorithm and used the GPU version of 2-opt to test its performance. The assumption was made that the initial solution s_0 is a random tour. We used a simple double-bridge move as a perturbation technique. Figure 11 presents the convergence of the Iterated Local Search with GPU 2-opt implementation. As for the other problems, the GPU algorithm gains more strength with the growth of instance size. Due to increased latency, the GPU ILS version does not give any substantial speedup over the CPU implementation in case of small problems (smaller than 200).

VI. CONCLUSION

In this paper we have presented a high-performance GPU implementation of 2-opt local search in Traveling Salesman Problem. We acknowledge that it may not be the fastest existing algorithm solving the TSP problem, however we believe that it is the fastest one implemented on GPU and provides a good base for more sophisticated approaches. The fastest sequential algorithms use complex pruning schemes and specialized data structures which we did not use. Instead, our algorithm solves the problem in a brute-force way, but due to the very high parallelism, the overall speed allows to tackle large TSP instances. The time needed to perform a simple local search operation can be decreased approximately 5 to 45 times compared to parallel CPU code implementation using 6 cores (The CPU parallel implementation based on OpenCL). The whole algorithm converges up to 20 times faster depending on the problem size (Figure 11), leading to very good solution in a very short time. We believe that the algorithm presents strong-scaling features, therefore we will try to parallelize it even further by using more CPUs and GPUs and possibly dividing the 2-opt task between multiple devices in order to effectively solve larger instances. The main contribution of this paper is the problem division scheme which allows to solve arbitrarily big problem instances using GPU as well as the techniques used to maintain data locality used to achieve very high performance. These optimization techniques reduce data movement, decrease read latency and use available compute resources instead of memory bandwidth.

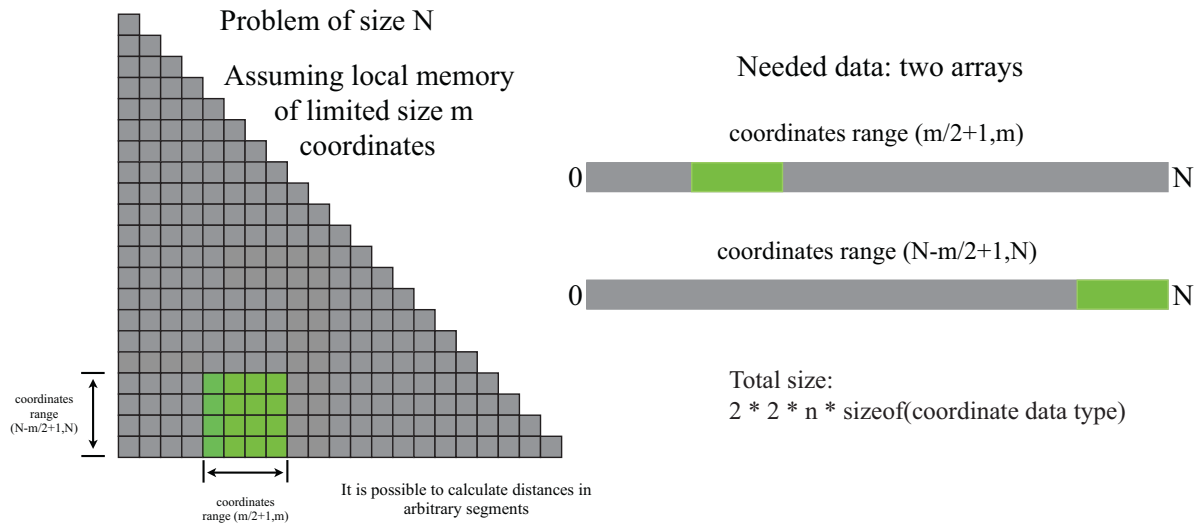


Figure 7. 2 subranges of coordinates are needed to compute local solution, assuming route ordered arrays of coordinates

```

1 int calculateDistance2D_extended(int i, int j, float2 * cA, float2 * cB) {
2     float dx, dy;
3     // 2 sets of coordinates needed, A for point i and B for point j
4     dx = cA[i].x - cB[j].x;
5     dy = cA[i].y - cB[j].y;
6     return (int)(sqrtf(dx * dx + dy * dy) + 0.5f);
7 }

```

Listing 2. A modified function with 2 coordinate sets used to calculate the distances between 2 points in large problems

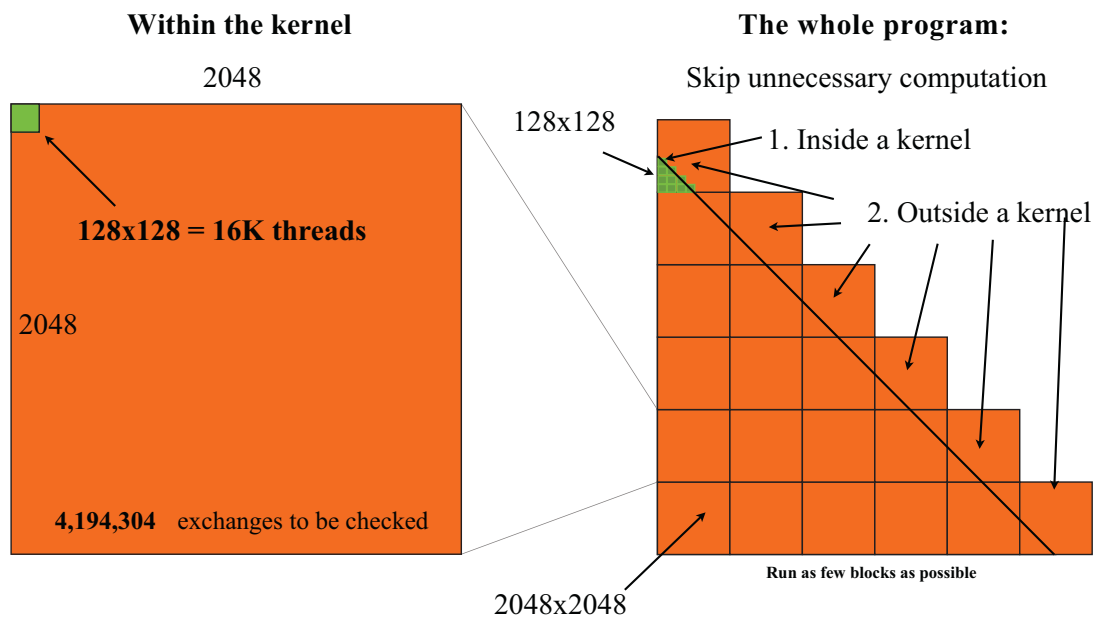


Figure 8. Big problems involve either multiple kernel launches or iteration inside a one kernel

Table II
2-OPT - TIME NEEDED FOR A SINGLE RUN, CUDA, GPU: GTX680, CPU: INTEL I7-3960X

Problem (TSPLIB)	GPU kernel time full 2-opt	Host to device copy time	Device to host copy time	GPU total time 2-opt	2-opt checks/s (Millions)	Time to first minimum from MF	Initial Length	Optimized Length
berlin52	20 μs	50 μs	11 μs	81 μs	76.56	687 μs	9951	8930
kroE100	21 μs	50 μs	11 μs	82 μs	282.92	897 μs	24846	23025
ch130	21 μs	50 μs	11 μs	82 μs	483.81	1183 μs	7849	7041
ch150	23 μs	50 μs	11 μs	84 μs	591.2	1342 μs	7742	7120
kroA200	24 μs	50 μs	11 μs	85 μs	1015.78	2.547 ms	34601	31685
ts225	24 μs	50 μs	11 μs	85 μs	1145.97	1.090 ms	135193	128513
pr299	26 μs	50 μs	11 μs	87 μs	1878.46	2.555 ms	61493	54895
pr439	32 μs	50 μs	11 μs	93 μs	3307.85	3.067 ms	124127	115490
rat783	53 μs	51 μs	11 μs	115 μs	6385.53	8.827 ms	10734	9658
vm1084	80 μs	51 μs	11 μs	142 μs	8122.51	11.862 ms	287710	267210
pr2392	299 μs	53 μs	11 μs	363 μs	11065.33	89.577 ms	454068	412085
pcb3038	471 μs	55 μs	11 μs	547 μs	10689.4	168.02 ms	165688	147690
fl3795	723 μs	54 μs	11 μs	788 μs	10529.32	256.19 ms	34843	31312
fnl4461	746 μs	58 μs	11 μs	815 μs	14098.03	327.1 ms	215085	194746
ri5934	1009 μs	59 μs	11 μs	1079 μs	18172.88	291.09 ms	627417	582958
pla7397	1547 μs	58 μs	11 μs	1616 μs	18153.6	870.2 ms	26954302	24734292
usa13509	4728 μs	66 μs	11 μs	4805 μs	19481.58	6.5251 s	23271693	20984503
d15112	5963 μs	69 μs	11 μs	6043 μs	19272.07	8.6757 s	1810355	1652806
d18512	8928 μs	75 μs	11 μs	9014 μs	19268.93	14.975 s	745983	675638
sw24978	14.99 ms	85 μs	11 μs	15.08 ms	20863.46	37.284 s	1002187	908598
pla33810	26.81 ms	96 μs	11 μs	26.92 ms	21340.36	68.67 s	76680735	69763154
pla85900	168.4 ms	205 μs	11 μs	168.6 ms	21915.11	1109.2 s	163831861	149708033
sra104815	249.5 ms	249 μs	11 μs	249.8 ms	22017.38	2093.4 s	291403	264889
usa115475	302.3 ms	287 μs	11 μs	302.6 ms	22054.81	3337.8 s	7143419	6492848
ara238025	1.2934 s	740 μs	11 μs	1.2941 s	21902.18	26497 s	674838	610795
lra498378	5.6369 s	1774 μs	11 μs	5.6388 s	22031.65	3078 m	2467227	2264810
lrb744710	12.634 s	2833 μs	11 μs	12.638 s	21947.17	203.8 h	1867266	1692308

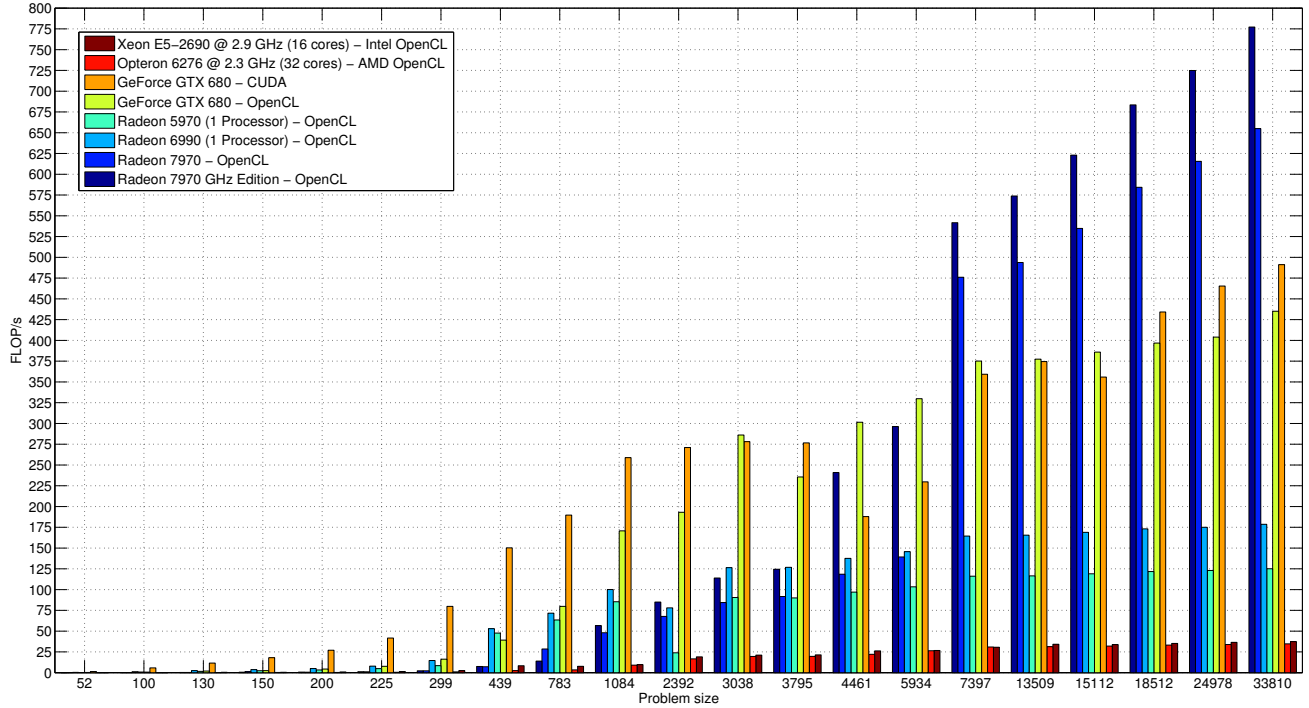


Figure 9. GFLOP/s (distance calculation) observed during the run using CUDA and OpenCL

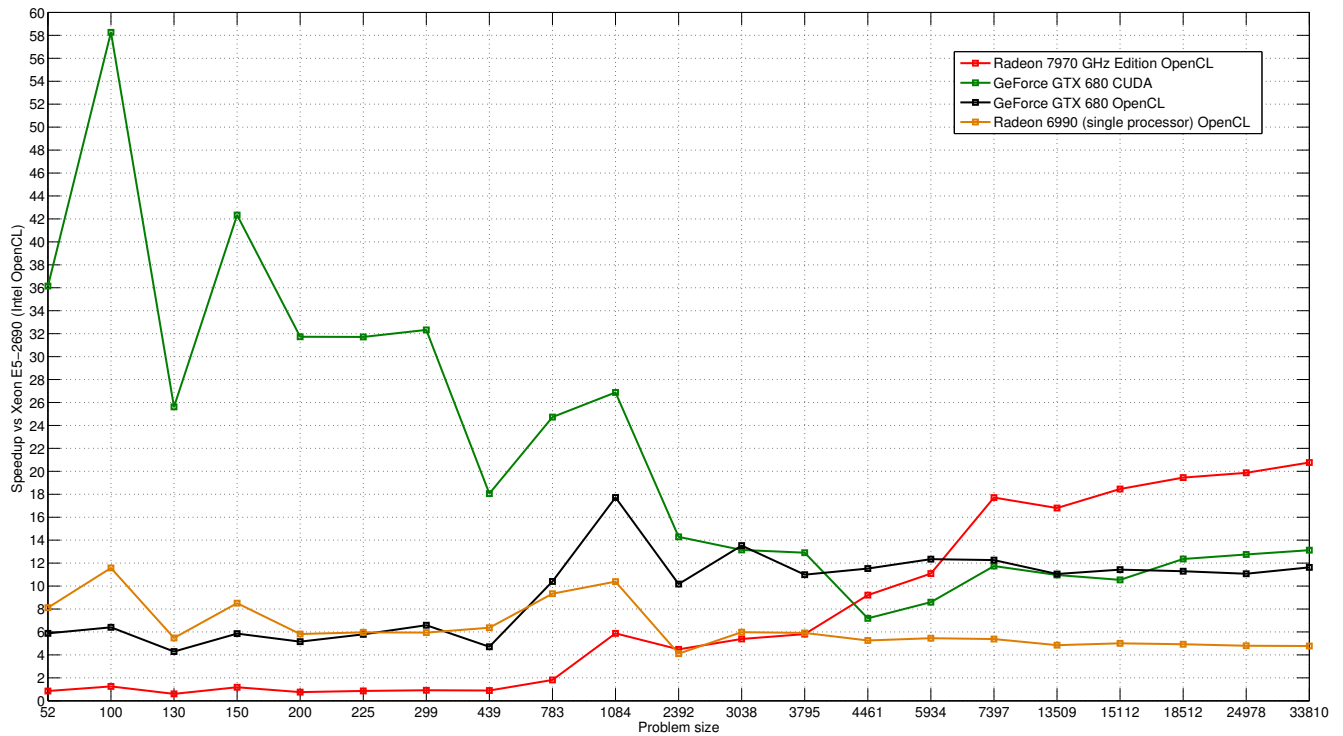


Figure 10. Speedup of the algorithm compared to the OpenCL parallel CPU implementation running on Intel Xeon E5-2690 (2 x 6 cores @ 2.93GHz)

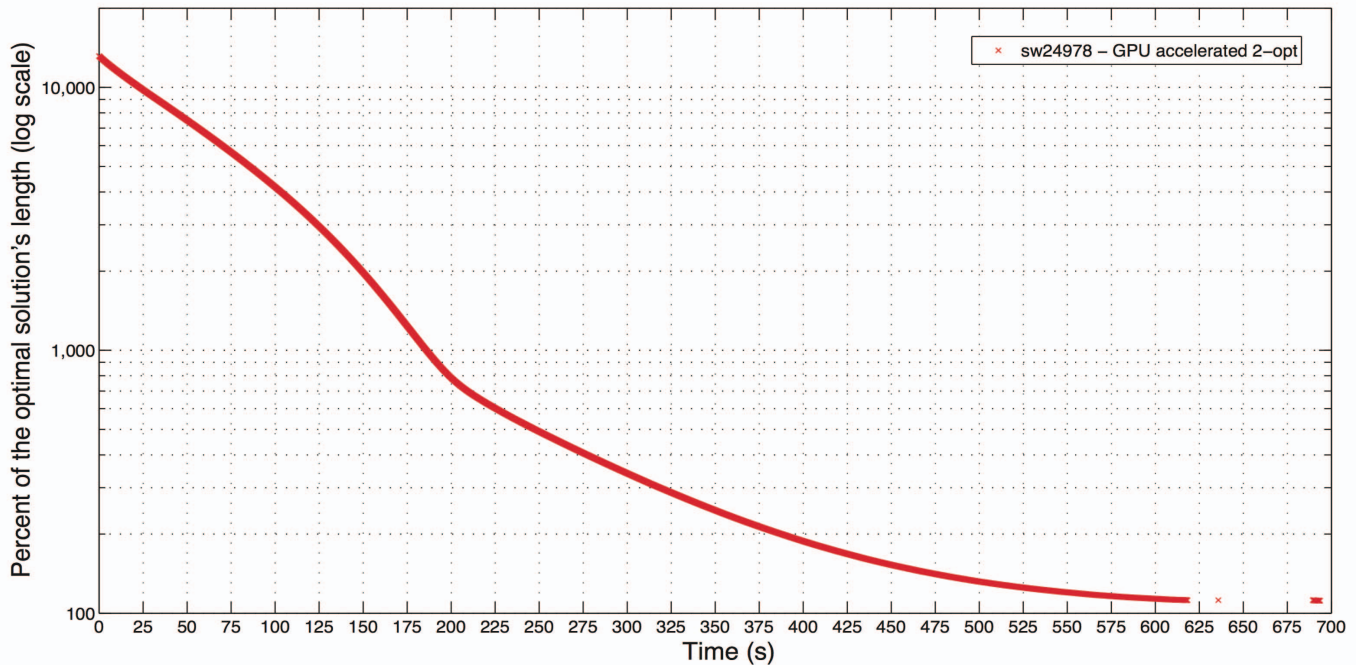


Figure 11. Iterated Local Search Convergence Speed (GPU) - sw24978.tsp

VII. FUTURE WORK

Our future work is to efficiently implement more complex local search algorithms such as 2.5-opt, 3-opt and Lin-Kernighan. However, these require much more effort in order to be parallelized. Also, simple ideas such as neighborhood pruning can be applied at the cost of the quality of the solution.

ACKNOWLEDGMENT

This work was supported by Core Research of Evolutional Science and Technology (CREST) project of Japan Science and Technology Agency (JST) and Grant-in-Aid for Scientific Research of MEXT Japan.

REFERENCES

- [1] Applegate, D.L., Bixby, R.E., Chvatal, V., Cook, W.J.: The Traveling Salesman Problem: A Computational Study. Princeton University Press, Princeton, 2007.
- [2] Lawler, E.L., Lenstra, J.K., Rinnooy Kan, A.H.G., Shmoys, D.B.: The Traveling Salesman Problem: A Guided Tour of Combinatorial Optimization. Wiley, Chichester, 1985.
- [3] Johnson, D. and McGeoch, L.: The Traveling Salesman Problem: A Case Study in Local Optimization. Local Search in Combinatorial Optimization, by E. Aarts and J. Lenstra (Eds.), pp. 215-310. London: John Wiley and Sons, 1997.
- [4] Garey, M.R. and Johnson, D.S. Computers and Intractability: A Guide to the Theory of NP-Completeness. San Francisco: W.H. Freeman, 1979.
- [5] Croes G. A.; A Method for Solving Traveling-Salesman Problems, Operations Research November/December 1958.
- [6] M. A. O'Neil, D. Tamir, and M. Burtcher.: A Parallel GPU Version of the Traveling Salesman Problem. 2011 International Conference on Parallel and Distributed Processing Techniques and Applications, pp. 348-353. July 2011.
- [7] Dorigo, M. and Gambardella, L.M.: Ant Colony System: A Cooperative Learning Approach to the Traveling Salesman Problem. IEEE Transactions on Evolutionary Computation, Vol. 1, No. 1, pp. 53-66. April 1997.
- [8] Fujimoto, N. and Tsutsui, S.: A Highly-Parallel TSP Solver for a GPU Computing Platform. Lecture Notes in Computer Science, Vol. 6046, pp. 264-271, 2011.
- [9] Reinelt, G.: TSPLIB - A Traveling Salesman Problem Library. ORSA Journal on Computing, Vol. 3, No. 4, pp. 376-384. Fall 1991.
- [10] Rego, C. and Glover, F.: Local Search and Metaheuristics. The Traveling Salesman Problem and its Variations, by G. Gutin and A.P. Punnen (Eds.), pp. 309-368. Dordrecht: Kluwer Academic Publishers, 2002.
- [11] Lourenco, H. R. Martin, O. C. Stutzle, T.: Iterated Local Search, International series in operations research and management science, 2003.
- [12] Karp, R. Reducibility among combinatorial problems: In Complexity of Computer Computations. Plenum Press, pp. 85-103. New York, 1972.
- [13] Tsai, H.; Yang, J. Kao, C. Solving traveling salesman problems by combining global and local search mechanisms, Proceedings of the 2002 Congress on Evolutionary Computation (CEC'02), 2002.
- [14] Pepper J.; Golden, B. Wasil, E. Solving the travelling salesman problem with annealing-based heuristics: a computational study. IEEE Transactions on Man and Cybernetics Systems, Part A, Vol. 32, No.1, pp. 72-77, 2002.
- [15] NVIDIA CUDA Programming Guide <http://docs.nvidia.com/cuda/index.html>
- [16] Helsgaun, K.; An Effective Implementation of the Lin-Kernighan Traveling Salesman Heuristic, European Journal of Operational Research, 2000.
- [17] Nilsson, Ch.; Heuristics for the Traveling Salesman Problem, Linkoping University
- [18] Bentley, J.; Experiments on traveling salesman heuristics, Proceedings of the first annual ACM-SIAM symposium on Discrete algorithms, SODA 1990.
- [19] <http://www.tsp.gatech.edu/concorde.html>
- [20] Luong T.V., Melab N., Talbi E.-G.; GPU Computing for Parallel Local Search Metaheuristic Algorithms. IEEE Transactions on Computers, Vol. 62, No. 1, Jan. 2013.